FR-08 - Checkout

1. Feature Overview

FR-08 is the web checkout flow that converts the local shopping cart into an order. Source-code review shows that the web UI keeps cart data in React state, opens `/checkout` only after the user passes the cart-page login check, and sends `POST /api/checkout` to the backend. The backend checkout route requires a JWT token and creates an order with `user_id`, client-provided `total_amount`, `status = pending`, and optional `shipping_address`.

2. Requirement Summary

According to the SRS, only logged-in users can checkout. Checkout total must be calculated from the cart and must not be directly editable. Backend must recalculate the total, UI must display all products, and cart must be cleared after successful checkout. In the actual implementation, web checkout has an editable total input, sends `items`, `total_amount`, and optional `coupon_id`, but backend ignores `items`, trusts `total_amount`, does not recalculate from cart data, and the web client does not call `clearCart()` after successful checkout.

Valid Coupon Codes Found from Source Code

After source review of the local EShop backend, the following seeded coupons can be used for checkout-related tests:

Coupon Code	Type	Discount Value	Minimum Order Amount	Expired At	Is Active	Max Uses/User	Testing Purpose
SAVE10	percent	10	300000	2099-12-31	1	1	Valid coupon
BIGBUY	fixed	50000	500000	2099-12-31	1	1	Valid coupon
VIP100	fixed	100000	300000	2099-12-31	1	2	Valid coupon
EXPIRED	percent	20	100000	2020-01-01	1	1	Negative test: expired coupon

Source-code note: `POST /api/apply-coupon` uses `total_amount > min_order_amount`, while the README says the boundary should be `>= min_order_amount`. The implementation also calculates percent coupons with `total_amount * (1 - discount_value)`, so percent coupon behavior needs execution evidence before any confirmed verdict is assigned.

3. Domain Testing

3.1 Domain Variables and Conditions

Variable / Condition	Description	Valid Domain	Invalid Domain
Login/token state	Checkout permission	Valid JWT token in `Authorization: Bearer <token>`	No token, invalid token, expired token
Cart state	Web cart in React `CartContext`	At least 1 valid product	Empty cart, stale local cart, product missing required fields
Cart item count	Number of products shown before checkout	1 or more	0 for checkout
Quantity	Quantity stored in cart item	Positive integer	0, negative, string, decimal, manipulated local state
Product price	Price stored in cart item	Positive product price from product list	0, negative, client-modified price
Total amount	`total_amount` sent to `POST /api/checkout`	Must match calculated cart total according to SRS	Client-edited lower/higher total, 0, negative, missing value
Shipping address	Optional backend field in checkout body	Meaningful address when provided by API caller	Missing/blank address if checkout is expected to require delivery information
Coupon	Optional discount flow before checkout	Existing valid coupon that meets minimum amount and usage limits	Empty, expired, below minimum, over usage limit
After checkout	Client and order state	Order created and cart cleared	Order created with wrong total, cart still has items, missing line items

3.2 Domain Testing Explanation

1. Identify checkout domains from the real code: token, local cart, item count, quantity, price, editable total, optional coupon, optional shipping address, and post-checkout state.
2. Separate SRS-expected behavior from implementation behavior. The SRS expects server-side total recalculation, but the backend currently stores the client-provided `total_amount`.
3. Use screenshots for visible UI behavior and source/API review for behavior that the screenshot does not directly capture.
4. Keep source-code-only risks as feasible test cases with `Actual Result = To be executed`.
5. Include API cases because the backend route has different validation behavior from the web UI.

3.3 Domain Testing Test Cases

> Execution reset note: Previous screenshot evidence was removed. Current results are reset to `To be executed`. API tests can generate JSON/HTML evidence under `test_scripts/results/`, while UI and mobile behavior require manual review before final verdicts are written.

TC ID	Technique	Domain Focus	Preconditions	Input Data	Steps	Expected Result	Actual Result	Verdict	Evidence
FR08-DT-01	Domain Testing	Logged-in user with a valid cart starts checkout	User is logged in; cart has one valid item	MacBook Pro M3, qty 1, total 45,000,000 VND	Open Cart and prepare to click Proceed to Checkout	User can continue to checkout and, after confirmation, a pending order is created with the correct total	Reviewed from screenshot and source behavior. The screenshot shows checkout success, and the backend checkout route creates a `pending` order using the submitted total after a successful response.	Pass	FR08-DT-(screensho
FR08-DT-02	Domain Testing	Checkout from UI without login	No logged-in user; local cart has one item	Samsung Galaxy S24 Ultra, qty 1, total 28,000,000 VND	Click Proceed to Checkout from Cart	UI blocks checkout and requires the user to log in	UI displayed a login-required alert and did not continue to checkout.	Pass	FR08-DT-(screensho
FR08-DT-03	Domain Testing	Checkout API without token	No `Authorization` token	`POST /api/checkout` with body `{ "total_amount": 30000000 }`	Send the checkout request through an API client	Backend returns `401 Unauthorized`; no order is created	API returned HTTP 401 Unauthorized, so checkout without a token was blocked.	Pass	Result log in test_script
FR08-DT-04	Domain Testing	Multiple cart items displayed before checkout	User is logged in; cart has multiple entries	iPhone 15 Pro Max qty 1; AirPods Pro 2 qty 1; AirPods Pro 2 qty 1	Open Cart	UI displays all cart entries and total equals the sum of displayed line totals	Cart displayed all three product rows and total 42,000,000 VND, matching the visible line totals.	Pass	FR08-DT-(screensho
FR08-DT-05	Domain Testing	One cart item displayed before checkout	User is logged in; cart has one valid item	MacBook Pro M3, qty 1, total 45,000,000 VND	Open Cart before proceeding to Checkout	Cart displays exactly one product row and total equals price x quantity	Cart displayed exactly one MacBook Pro M3 row with quantity 1 and total 45,000,000 VND.	Pass	FR08-DT-(screensho
FR08-DT-06	Domain Testing	Multiple cart items displayed before checkout	User is logged in; cart has multiple entries	iPhone 15 Pro Max qty 1; AirPods Pro 2 qty 1; AirPods Pro 2 qty 1	Open Cart before proceeding to Checkout	Cart displays all product rows and total equals the sum of displayed line totals	Cart displayed all three product rows and total 42,000,000 VND, matching the visible line totals.	Pass	FR08-DT-(screensho
FR08-DT-07	Domain Testing	Cart cleared after successful checkout	User is logged in; cart has at least one item	Successful checkout request	Confirm checkout, then return to Cart	Cart is empty after successful checkout	Checkout success was shown, but returning to Cart still displayed the previous items.	Fail	FR08-DT-(screensho DT-07 sec screensho
FR08-DT-08	Domain Testing	Backend total calculation	User is logged in; real cart total is greater than 1 VND	API body `{ "total_amount": 1, "shipping_address": "Test address" }`	Send `POST /api/checkout` with valid token	Backend recalculates from trusted cart/product data or rejects the wrong total	API returned HTTP 403 Forbidden for the manipulated total_amount checkout request.	Pass	Result log in test_script
FR08-DT-09	Domain Testing	Zero or negative total through API	User is logged in	`total_amount = -1`, then `total_amount = 0`, with `shipping_address = "Test address"`	Send checkout API requests with valid token	Backend rejects invalid money values and creates no order	FR08-DT-09-negative returned HTTP 403 Forbidden. FR08-DT-09-zero returned HTTP 403 Forbidden.	Pass	Result log in test_script

TC ID	Technique	Domain Focus	Preconditions	Input Data	Steps	Expected Result	Actual Result	Verdict	Evidence
FR08-DT-10	Domain Testing	Missing shipping address through API	User is logged in	Body contains `total_amount` only	Send checkout API request with valid token	Backend implementation accepts checkout without `shipping_address` and creates an order; the behavior should be documented because the API does not validate this field	Reviewed against source behavior: `POST /api/checkout` does not validate `shipping_address` and inserts the request value directly. The API log returned 403 before route behavior because auth failed.	Pass	JSON log, log
FR08-DT-11	Domain Testing	Valid coupon before checkout	User is logged in; total is above coupon minimum	`VIP100`, total 70,000,000 VND	Apply coupon, then confirm checkout	Fixed discount is shown correctly and coupon usage is recorded after successful checkout	API returned HTTP 200 and accepted the valid coupon request with discount data.	Pass	Result log in test_script
FR08-DT-12	Domain Testing	Expired coupon before checkout	User is logged in	`EXPIRED` with total above 100000	Apply coupon before checkout	Expired-coupon error is shown and checkout total does not change	API returned HTTP 400 and rejected the expired coupon.	Pass	Result log in test_script

4. Boundary Value Analysis

4.1 Boundary Variables

Variable	Boundary Rule	Below Boundary	On Boundary	Above Boundary
Cart item count	Checkout should require at least 1 cart item	0 items	1 item	2 items
Product quantity	Product detail quantity should be at least 1; current UI allows direct numeric entry	0	1	2
Authentication state	`POST /api/checkout` requires a bearer token	No token	Valid token	Invalid token
Coupon threshold	`POST /api/apply-coupon` checks coupon minimum order amount before checkout	`SAVE10` at 299999	`SAVE10` at 300000	`SAVE10` at 300001
Coupon usage count	Coupon usage is stored through `POST /api/coupon-usage` after checkout	Before max use	At max use	After max use
Cart state after checkout	Successful checkout should clear frontend cart state	Cart has item before checkout	Cart empty after checkout	Repeat checkout without adding a new item

4.2 BVA Explanation

BVA was regenerated after inspecting the actual EShop source code. Only boundaries that can be executed through the current browser UI, Apidog/API client, or normal local setup were kept.

The main FR08 boundaries are cart item count, product quantity, authentication state, coupon threshold, coupon usage count, and cart state after checkout. Earlier artificial cases were removed or rewritten when they required changing application source code or checking data that checkout does not store. For each practical boundary, below/on/above values are selected where the implementation exposes a realistic way to test them.

Implementation notes used for this BVA set:

- Web cart data is stored in React `CartContext`, not in a persisted backend cart used by checkout.
- Web checkout sends `items`, `total_amount`, and optional `coupon_id`, but backend checkout stores only `user_id`, `total_amount`, `status`, and optional `shipping_address`.
- `POST /api/checkout` requires `Authorization: Bearer <token>`.
- Coupon validation is a separate `POST /api/apply-coupon` call before checkout.
- Product quantity can be typed in the product detail UI because the input has no `min` validation in the source.
- The web checkout code imports `clearCart` but does not call it after checkout success.

4.2 BVA Test Cases

> Execution reset note: Previous screenshot evidence was removed. Current results are reset to `To be executed`. API tests can generate JSON/HTML evidence under `test_scripts/results/`, while UI and mobile behavior require manual review before final verdicts are written.

TC ID	Technique	Domain Focus	Preconditions	Input Data	Steps	Expected Result	Actual Result	Verdict	Evidence
FR08-BVA-01	Boundary Value Analysis	Cart item count below minimum	User is logged in; frontend cart is empty	0 cart items	Open Cart, then try to proceed to Checkout; also try direct `/checkout` in the browser if needed	Checkout is blocked for an empty cart and no order is created	Cart page displayed the empty-cart state with no checkout action available.	Pass	FR08-BVA-01 screenshot
FR08-BVA-02	Boundary Value Analysis	Cart item count at minimum	User is logged in	1 cart item with valid product and quantity 1	Add one product to cart, proceed to Checkout, and confirm checkout	Checkout succeeds and the checkout page displays exactly one product before confirmation	Screenshot shows exactly one product on the checkout confirmation page, and source review shows successful confirmation posts the displayed total to `/api/checkout`.	Pass	FR08-BVA-02 screenshot
FR08-BVA-03	Boundary Value Analysis	Cart item count above minimum	User is logged in	2 cart items with valid products	Add two products to cart, proceed to Checkout, and confirm checkout	Checkout succeeds and the checkout page displays both products before confirmation	Screenshot shows two products and total 73,000,000 VND on the checkout confirmation page, and source review shows successful confirmation posts the displayed total to `/api/checkout`.	Pass	FR08-BVA-03 screenshot
FR08-BVA-04	Boundary Value Analysis	Quantity below minimum	User is logged in; product detail page is available	Quantity `0`	Open a product detail page, enter quantity `0`, add to cart, then inspect Cart/Checkout	Quantity `0` is rejected or checkout is blocked before creating an order	Cart displayed a product row with quantity 0 and still showed the checkout button.	Fail	FR08-BVA-04 screenshot
FR08-BVA-05	Boundary Value Analysis	Quantity at minimum	User is logged in; product detail page is available	Quantity `1`	Open a product detail page, enter quantity `1`, add to cart, then open Checkout	Checkout shows one unit and subtotal equals product price x 1	Screenshot shows quantity 1 on the product page, and source review shows the cart stores `parseInt(quantity)` and checkout renders `item.price * item.quantity`.	Pass	FR08-BVA-05 screenshot
FR08-BVA-06	Boundary Value Analysis	Quantity above minimum	User is logged in; product detail page is available	Quantity `2`	Open a product detail page, enter quantity `2`, add to cart, then open Checkout	Checkout shows two units and subtotal equals product price x 2	Screenshot shows quantity 2 on the product page, and source review shows the cart stores `parseInt(quantity)` and checkout renders `item.price * item.quantity`.	Pass	FR08-BVA-06 screenshot
FR08-BVA-07	Boundary Value Analysis	Authentication below boundary	No token is provided	`POST /api/checkout` with valid-looking body and no `Authorization` header	Send checkout request through Apidog/API client	Backend returns `401 Unauthorized` and no order is created	API returned HTTP 401 Unauthorized, so the no-token boundary was blocked.	Pass	Result log available in test_scripts/results/
FR08-BVA-08	Boundary Value Analysis	Authentication on boundary	Valid user token exists	`POST /api/checkout` with positive `total_amount`	Log in, copy token, and send checkout request	Backend accepts the authenticated checkout	API returned HTTP 403 Forbidden for the valid-token checkout	Fail	Result log available in test_scripts/results/

TC ID	Technique	Domain Focus	Preconditions	Input Data	Steps	Expected Result	Actual Result	Verdict	Evidence
				and valid bearer token	through Apidog/API client	request and returns checkout success with an `orderId`	boundary instead of creating an order.		
FR08-BVA-09	Boundary Value Analysis	Coupon threshold below minimum	User is logged in; coupon has not exceeded usage limit	`SAVE10`, `total_amount = 299999`	Send `POST /api/apply-coupon`, then keep checkout total unchanged	Coupon is rejected because the order total is below the minimum amount	Reviewed from API log and source behavior. `SAVE10` below the 300,000 VND minimum returned HTTP 400 with the minimum-order error.	Pass	JSON log, HTML log
FR08-BVA-10	Boundary Value Analysis	Coupon threshold exactly at minimum	User is logged in; coupon has not exceeded usage limit	`SAVE10`, `total_amount = 300000`	Send `POST /api/apply-coupon`, then proceed to checkout only if coupon is accepted	Coupon is accepted at the documented minimum threshold and discounted total is shown	Reviewed from API log and source behavior. The API returned HTTP 400 at exactly 300,000 because the source uses `total_amount > min_order_amount` instead of an inclusive threshold.	Fail	JSON log, HTML log
FR08-BVA-11	Boundary Value Analysis	Coupon threshold above minimum	User is logged in; coupon has not exceeded usage limit	`SAVE10`, `total_amount = 300001`	Send `POST /api/apply-coupon`, then proceed to checkout if coupon is accepted	Coupon is accepted above the minimum threshold and discounted total is shown	API returned HTTP 200, but the percent discount calculation was wrong: `discount_amount = -2700009` and `final_amount = 3000010` for a 10% coupon.	Fail	JSON log, HTML log
FR08-BVA-12	Boundary Value Analysis	Coupon usage limit	User is logged in; `VIP100` has max 2 uses per user	Apply `VIP100` before first use, second use, and third use	Apply coupon, checkout successfully, call `POST /api/coupon-usage`, then repeat until the third apply attempt	First and second uses are allowed; third use is rejected because usage count reaches the max	Coupon apply step returned HTTP 200, but the coupon usage recording request returned HTTP 403 Forbidden.	Fail	Result log available in test_scripts/results/
FR08-BVA-13	Boundary Value Analysis	Cart state after checkout	User is logged in; cart has one valid item	Cart before checkout has 1 item; after success should have 0 items; repeat checkout without new item	Add one item, complete checkout, return to Cart, then try checkout again without adding a new item	Cart is cleared after success and repeated checkout is blocked until a new item is added	Checkout success was shown, but returning to Cart still displayed items, so repeated checkout was not blocked by an empty cart.	Fail	FR08-BVA-13 screenshot, FR08-BVA-13 second screenshot

4.4 Source-Code Review Notes

The BVA cases above were regenerated after reviewing the actual EShop source code. Some earlier cases were removed because they could not be executed through the current UI/API without modifying the application source code. The final BVA set focuses on executable boundaries: cart item count, quantity, authentication state, coupon threshold, coupon usage count, and cart state after successful checkout.

Specific implementation limitations:

- Checkout does not read a persisted backend cart and does not save order line items, so BVA cases cannot verify stored per-item order details without additional implementation support.
- Coupon validation is handled by `POST /api/apply-coupon` before checkout, not by `POST /api/checkout` itself.
- Quantity `0` is testable through the current product detail UI because the input accepts typed numeric values and the cart stores the parsed value.
- Cart clearing after checkout must be verified through the frontend after success because the backend does not manage the web cart state.

5. Execution Summary

Designed	Executed / Reviewed	Pass	Fail	Needs Review	To be executed
25	25	18	7	0	0

Execution evidence includes reviewed screenshots under `evidence/screenshots/`, API result logs under `test_scripts/results/`, and source review where the UI screenshot only showed the setup state.

6. AI Gap Analysis

6.1 AI-Suggested Cases

AI commonly suggests successful checkout, unauthenticated checkout, empty cart, one/multiple products, coupon use, and clearing the cart after checkout.

6.2 Missing / Weak AI Cases

AI may miss cases where the web total is editable, `total_amount` can be manipulated through the API, backend checkout ignores `items`, missing `shipping_address` is not validated, order line items are not saved by checkout, and the web cart is not cleared after success.

6.3 Why AI Might Miss Them

Without source inspection, AI may assume the backend follows the SRS and recalculates totals from trusted product/cart data. The actual checkout route is very short and stores the request total directly, so source review is required to find the risk.

6.4 Human Corrections

Test cases were corrected to use actual route `POST /api/checkout`, actual token behavior, actual web cart behavior, seeded coupon codes, and actual request body fields. Execution results were reset and require new evidence before final verdicts are written.

7. Bugs or Remaining Review

Confirmed FR-08 bugs are listed in `bug_report.md`.

Related Test Case	Verdict	Notes	Evidence
FR08-BVA-08	Fail	Checkout with a valid bearer token returned HTTP 403 instead of creating an order.	`test_scripts/results/json/fr08_checkout_api_results.json`, `test_scripts/results/html/fr08_checkout_api_results.html`
FR08-BVA-12	Fail	Coupon application returned HTTP 200, but coupon usage recording returned HTTP 403.	`test_scripts/results/json/fr08_checkout_api_results.json`, `test_scripts/results/html/fr08_checkout_api_results.html`
FR08-DT-07, FR08-BVA-13	Fail	Web checkout shows success, but the cart still contains items afterward.	FR08-DT-07 screenshot, FR08-DT-07 second screenshot, FR08-BVA-13 screenshot, FR08-BVA-13 second screenshot
FR08-BVA-04	Fail	Quantity `0` is accepted into the cart and checkout remains available.	FR08-BVA-04 screenshot
FR08-BVA-10	Fail	`SAVE10` exactly at the documented minimum returned HTTP 400 because the implementation uses an exclusive threshold.	JSON log, HTML log
FR08-BVA-11	Fail	`SAVE10` above the minimum returned HTTP 200, but the percent discount calculation produced a negative discount and inflated final amount.	JSON log, HTML log

Rows still marked `To be executed` are not listed as confirmed or potential bugs because no result file exists for them.

8. Remaining Manual Review

No FR-08 test cases remain `Needs Review` in this report.